Le microservice Student Expression Service

Projet FreeSpace — Document de conception architecturale et fonctionnelle

Le microservice **Student Expression Service** gère l'espace d'expression communautaire des étudiants. Il permet de créer des publications (expériences universitaires, questions, conseils et discussions) en mode nominatif ou anonyme, d'ajouter des commentaires imbriqués, de réagir aux contenus et de signaler les publications ou commentaires inappropriés.

Afin d'éviter le couplage fort et les pannes en cascade, ce service fonctionne de manière autonome sans lien direct avec les services de réservation de salles, d'espaces d'étude, de groupes ou d'objets trouvés. Il communique uniquement avec **User Management Service** et **Notification Service**.

1. Conception du microservice : diagramme de classes

Le service repose sur quatre entités principales (`Publication`, `Commentaire`, `Reaction` et `Signalement`) ainsi que deux énumérations (`TypePublication` et `TypeReaction`). Les références aux étudiants sont conservées via leurs identifiants (`auteurId`, `utilisateurId`) sans dupliquer la table des utilisateurs.

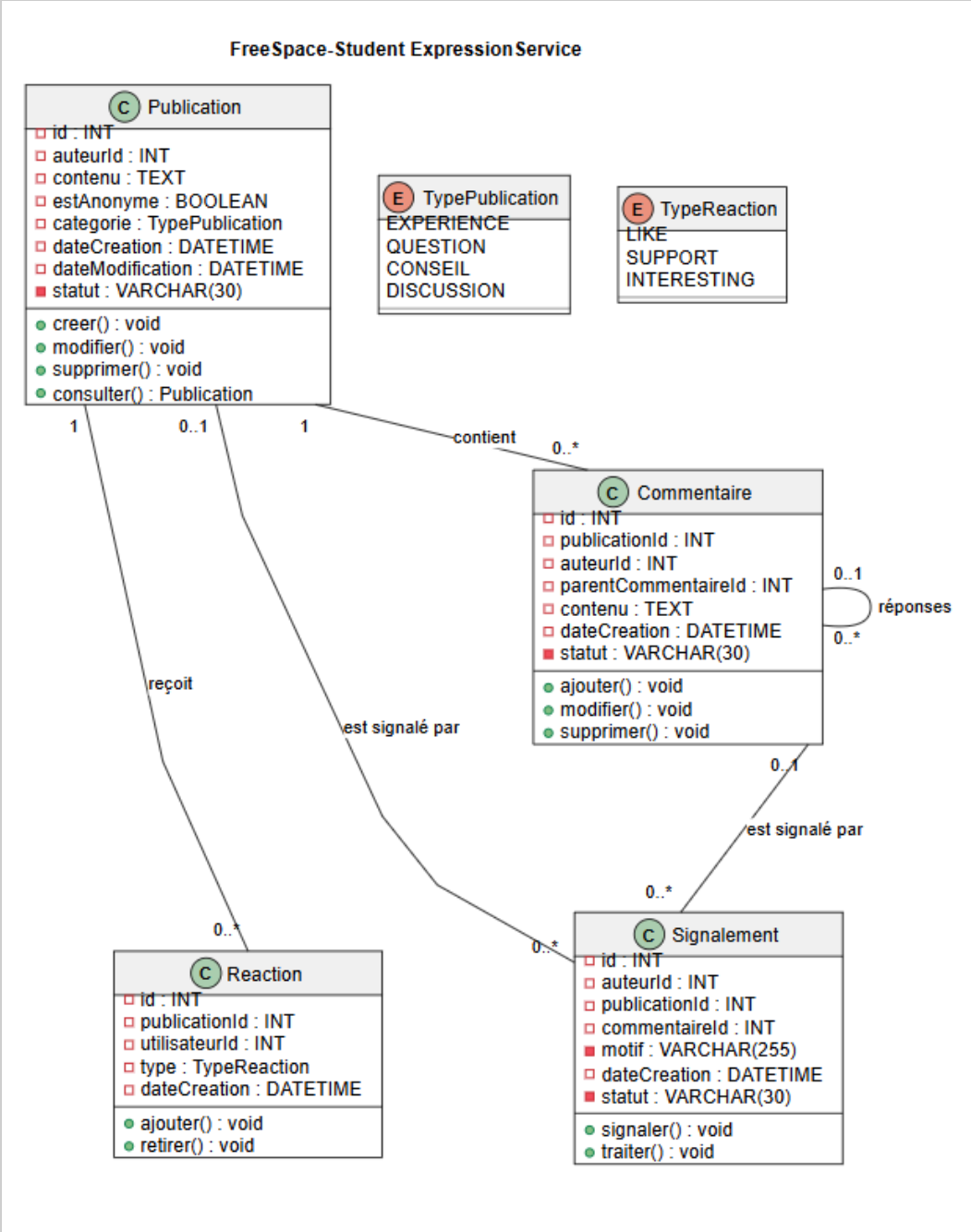


Figure 1 — Diagramme de classes du Student Expression Service et ses énumérations.

2. Interactions avec les autres microservices

Les échanges externes sont limités aux besoins strictement indispensables, en séparant les appels synchrones REST de la diffusion asynchrone via RabbitMQ.

2.1 Communication asynchrone : RabbitMQ

Après enregistrement en base de données, le service publie des événements métier dans RabbitMQ. Le **Notification Service** consomme ces messages pour informer les utilisateurs concernés sans bloquer le traitement des requêtes.

Événement publié	Déclenchement métier	Destinataire & Action
`PostCommented`	Ajout d'un commentaire ou d'une réponse sur une publication	Notification envoyée à l'auteur de la publication (ou du commentaire parent).
`PostReacted`	Attribution d'une réaction (`LIKE`, `SUPPORT`, `INTERESTING`)	Notification envoyée à l'auteur de la publication concernée.
`ContentReported`	Création d'un signalement sur une publication ou un commentaire	Alerte transmise aux administrateurs pour examen et modération.

2.2 Communication synchrone : REST

1. **Exposition API (Frontend → API Gateway → Expression Service) :** Le client appelle les endpoints REST sécurisés par jeton JWT pour consulter le fil, publier, commenter, réagir ou signaler.
2. **Résolution d'identité (`GET /api/users/{id}` vers User Management) :** Utilisé uniquement pour récupérer le nom et le prénom de l'étudiant lorsque `estAnonyme = false`, ou pour vérifier le rôle Administrateur lors du traitement d'un signalement.

2.3 Vue d'ensemble des interactions

Émetteur	Destinataire	Mode / Protocole	Action réalisée
Frontend	API Gateway	Synchrone — REST + JWT	Transmettre les requêtes utilisateur authentifiées.
API Gateway	Expression Service	Synchrone — REST	Router les appels vers `/api/publications/**` et `/api/signalements/**`.
Expression Service	PostgreSQL	JDBC / SQL	Enregistrer publications, commentaires, réactions et signalements.
Expression Service	User Management	Synchrone — REST	Récupérer le nom de l'auteur non anonyme et vérifier les droits Admin.
Expression Service	RabbitMQ	Asynchrone — AMQP	Publier `PostCommented`, `PostReacted` et `ContentReported`.

2.4 Contrat minimal d'un événement

Champ	Signification
`eventId`	Identifiant unique (UUID) permettant d'éviter les doublons côté consommateur.
`sourceService`	Nom du microservice émetteur (`student-expression-service`).
`eventType`	Type d'événement (`PostCommented`, `PostReacted`, `ContentReported`).

Champ	Signification
`occurredAt`	Date et heure de génération de l'événement.
`recipientIds`	Liste des identifiants des destinataires ciblés (auteur ou administrateurs).
`payload`	Données utiles à la notification (`publicationId`, extrait du message, type de réaction).

2.5 Gestion des erreurs et des pannes

Situation	Comportement prévu
`Notification Service` indisponible	Les opérations d'écriture aboutissent normalement ; les messages restent en attente dans RabbitMQ.
`User Management` lent ou hors ligne	Bascule en mode dégradé (affichage d'un libellé générique) sans bloquer la lecture du fil.
Double soumission d'une réaction	Contrainte d'unicité en base sur `(publicationId, utilisateurId)` pour éviter les doublons.
Erreur de publication RabbitMQ	Journalisation de l'erreur et nouvelle tentative d'envoi.

3. Contrats d'API — Student Expression Service

Ces endpoints REST sont exposés derrière l'API Gateway et protégés par jeton JWT.

Méthode	Endpoint	Fonction	Réponse
POST	`/api/publications`	Créer une publication (nominative ou anonyme)	201
GET	`/api/publications`	Lister les publications (filtrable par catégorie)	200
GET	`/api/publications/{id}`	Consulter une publication spécifique	200 / 404
PUT	`/api/publications/{id}`	Modifier une publication existante	200 / 403
DELETE	`/api/publications/{id}`	Supprimer une publication	204 / 403
POST	`/api/publications/{id}/commentaires`	Ajouter un commentaire ou une réponse	201
GET	`/api/publications/{id}/commentaires`	Lister les commentaires d'une publication	200
POST	`/api/publications/{id}/reactions`	Ajouter ou modifier une réaction	201
DELETE	`/api/publications/{id}/reactions`	Retirer sa réaction d'une publication	204

Méthode	Endpoint	Fonction	Réponse
POST	`/api/signalements`	Signaler une publication ou un commentaire	201
PATCH	`/api/signalements/{id}/statut`	Traiter un signalement (réservé Administrateur)	200 / 403

4. Interactions avec l'infrastructure

Composant	Rôle
API Gateway	Route les requêtes `/api/publications/**` et `/api/signalements/**` vers le service.
Eureka	Permet l'enregistrement et la découverte dynamique du service.
Config Server	Fournit la configuration centralisée (base PostgreSQL, RabbitMQ, sécurité JWT).
RabbitMQ	Assure la transmission asynchrone des événements vers le service de notification.
PostgreSQL	Stocke les données relationnelles dans la base dédiée `expression_db`.

5. Choix de PostgreSQL

PostgreSQL est choisi pour gérer la base `expression_db` (tables `publications`, `commentaires`, `reactions` et `signalements`) pour les raisons suivantes :

- **Intégrité relationnelle :** Gestion native des clés étrangères et des réponses imbriquées via `parent_commentaire_id`.
- **Transactions ACID :** Cohérence garantie lors du traitement des signalements et de la modération des contenus.
- **Contraintes d'unicité :** Empêche un étudiant d'enregistrer plusieurs réactions simultanées sur une même publication.

6. Énumérations

Énumération	Valeurs
`TypePublication`	`EXPERIENCE`, `QUESTION`, `CONSEIL`, `DISCUSSION`
`TypeReaction`	`LIKE`, `SUPPORT`, `INTERESTING`

7. Règles de fonctionnement

- **Anonymat :** Lorsque `estAnonyme` vaut `true`, l'identifiant `auteurId` est conservé uniquement en base pour la modération et n'est jamais renvoyé par l'API publique.
- **Droits de modification :** Un étudiant ne peut modifier ou supprimer que ses propres publications et commentaires.
- **Signalements :** Un signalement porte soit sur une publication (`publicationId`), soit sur un commentaire (`commentaireId`). Seul un administrateur peut exécuter `traiter()`.